

Deep_Reinforcement_Partie1

Présentation des algorithmes implémentés

1. Programmation dynamique

Policy Evaluation

L'algorithme d'évaluation itérative de politique permet d'estimer la **valeur d'un état** $V(s)$ sous une politique donnée π .

Cette valeur représente la récompense moyenne cumulée que l'agent peut espérer obtenir en partant de cet état et en suivant la politique.

L'algorithme met à jour les valeurs $V(s)$ de manière successive à l'aide de la **relation de Bellman** en prenant en compte toutes les actions possibles a , pondérées par leur probabilité $\pi(a|s)$, ainsi que tous les états suivants s' et les récompenses possibles r , pondérés par les probabilités de transition $p(s', r | s, a)$.

Le processus se poursuit jusqu'à **convergence**, c'est-à-dire tant que la variation maximale entre deux itérations successives reste supérieure à un seuil prédéfini.

Les **états terminaux** sont traités comme des points fixes : leur valeur reste constante à zéro, puisqu'aucune récompense future n'est attendue une fois ces états atteints.

Cette méthode est applicable à tout environnement défini par :

- un ensemble d'états S
- un ensemble d'actions A
- une fonction de transition probabiliste p
- et une politique π .

Expérimentation : Policy Evaluation sur GridWorld

Environnement : GridWorld

Paramètres :

- $\theta = 0.0001$
- $\gamma = 0.99$
- États terminaux : reward = -3 (haut droite), reward = +1 (bas droite)

Analyse de la politique : Always Up

Valeurs sous politique always up :

```
[ 0.00525078  0.00983544  0.00846198  0.00828474  0.          0.00519827
 0.00973708  0.00837736  0.00820189 -3.          0.00514629  0.00963971
 0.00829358  0.00811987 -2.97          0.00509482  0.00954331  0.00821065
 0.00803867 -2.9403          0.00504387  0.00944788  0.00812854  0.00795829
 0.          ]
```

- Les valeurs des états sont très faibles, proches de 0.
- L'agent remonte dans la grille sans jamais atteindre les états terminaux positifs.
- Dans certains cas, il atteint l'état terminal négatif en haut à droite (récompense -3).
- Ce comportement est cohérent : monter ne mène jamais à la récompense +1.

Analyse de la politique : Always down

Valeurs sous politique always down :

```
[1.46689721e-03 2.18734964e-03 6.00569225e-04 9.89557815e-03
 0.00000000e+00 1.46689721e-03 2.18734964e-03 6.00569225e-04
 9.89557815e-03 9.80100000e-01 1.46689721e-03 2.18734964e-03
 6.00569225e-04 9.89557815e-03 9.90000000e-01 1.46689721e-03
 2.18734964e-03 6.00569225e-04 9.89557815e-03 1.00000000e+00
 1.46689721e-03 2.18734964e-03 6.00569225e-04 9.89557815e-03
 0.00000000e+00]
```

- Valeurs élevées (~ 1.0) près du terminal à +1 (état 24).
- Politique très efficace pour atteindre le terminal positif.
- Preuve que cette stratégie maximise les récompenses dans ce cas précis.

Analyse de la politique : Always Right

Valeurs sous politique always right :

```
[-2.910897  -2.9403  -2.97  -3.          0.          0.00617781
 0.00617781  0.00617781  0.00617781  0.00617781  0.00987845  0.00987845
 0.00987845  0.00987845  0.00987845  0.00487831  0.00487831  0.00487831
 0.00487831  0.00487831  0.970299  0.9801  0.99  1.
 0.          ]
```

- Fortes valeurs négatives au début de la grille (proximité du terminal -3).
- Valeurs proches de +1 dans la dernière ligne : trajectoire efficace vers le terminal positif.
- Politique qui commence mal mais s'oriente bien vers la fin.

Analyse de la politique : Always Left

```
Valeurs sous politique always left :
[0.00807062 0.00798991 0.00791001 0.00783091 0.          0.00853604
 0.00845068 0.00836617 0.00828251 0.00819968 0.00155217 0.00153665
 0.00152129 0.00150607 0.00149101 0.00982681 0.00972854 0.00963126
 0.00953494 0.00943959 0.00447635 0.00443158 0.00438727 0.0043434
 0.          ]
```

- Valeurs faibles et relativement constantes.
- L'agent semble rester "bloqué" dans des zones sans terminal.
- Politique peu efficace : ne mène ni vers +1 ni vers -3.

Analyse de la politique : Uniforme aléatoire

```
Valeurs sous politique uniforme aléatoire :
[-0.85423978 -0.97546869 -1.26452911 -1.84640662 0.          -0.7679074
 -0.84728542 -1.02297337 -1.31907311 -1.69832108 -0.63349621 -0.65717309
 -0.70244833 -0.76196181 -0.81426743 -0.50126938 -0.47215904 -0.39617639
 -0.24292156 -0.01547894 -0.41866565 -0.35323394 -0.18328531 0.19205611
 0.          ]
```

- Valeurs majoritairement négatives.
- Indique que l'agent subit souvent la pénalité du terminal négatif.
- Quelques valeurs positives apparaissent quand la politique permet parfois d'atteindre +1.
- Comportement "aléatoire" donc résultats médiocres mais variables.

Conclusion Générale

- Les résultats sont cohérents avec les politiques définies.
- Always Down et Always Right sont les meilleures politiques pour atteindre l'état +1.
- Always Up conduit à l'état terminal -3, donc produit des valeurs faibles/négatives.
- Always Left garde l'agent dans une boucle improductive.
- La politique uniforme montre les limites d'un comportement sans objectif.

Policy Iteration

L'algorithme **Policy Iteration** repose sur deux étapes principales, exécutées de manière itérative :

1. l'évaluation de la politique actuelle,
2. son amélioration en sélectionnant les meilleures actions.

Policy Evaluation

On commence avec une **politique initiale** π choisie aléatoirement (dans le code : `pi = np.array([np.random.choice(A) for s in S])`).

L'objectif est de calculer les valeurs d'états $V(s)$ que l'agent obtiendrait en **suivant cette politique**, c'est-à-dire :

$$V(s) = \sum_{s', r} p(s, \pi(s), s', r) \cdot \left[r + \gamma V(s') \right]$$

Cette étape est implémentée dans une boucle `while`, où l'on met à jour chaque $V[s]$ jusqu'à ce que la variation maximale `delta` entre deux itérations devienne inférieure à un seuil `theta`.

dans le code Les **états terminaux** T sont initialisés avec une valeur nulle ($V[T] = 0.0$) car l'épisode se termine dès qu'on les atteint.

Policy Improvement

À partir des valeurs estimées, on met à jour la politique en choisissant, pour chaque état s , l'action a qui maximise l'espérance de retour. Si la politique ne change plus, l'algorithme s'arrête.

Expérimentation : Policy Iteration sur GridWorld

Paramètres utilisés pour le test

- **Environnement** : GridWorld 5x5
- **Récompenses** :
 - État terminal en haut à droite (état 4) : **-3.0**
 - État terminal en bas à droite (état 24) : **+1.0**
- **Facteur d'actualisation** : $\gamma=0.99$ `gamma = 0.99`
- **Seuil de convergence** : $\theta=10^{-4}$ `theta = 10^{-4}`

Politique optimale (action par état) :

[1 1 1 3 0 1 1 1 1 3 1 1 1 1 3 1 1 1 1 3 1 1 1 1 0]

Valeur des états sous cette politique :

0.93206535	0.94148015	0.95099005	0.96059601	0.	0.94148015
0.95099005	0.96059601	0.970299	0.9801	0.95099005	0.96059601
0.970299	0.9801	0.99	0.96059601	0.970299	0.9801
0.99	1.	0.970299	0.9801	0.99	1.
0.					

```
[1 1 1 3 0
 1 1 1 1 3
 1 1 1 1 3]
```

```
1 1 1 1 3
1 1 1 1 0]
```

Politique optimale trouvée

```
[→ → → ↓ x
→ → → → ↓
→ → → → ↓
→ → → → ↓
→ → → → x]
```

- L'agent se déplace vers la **droite**, puis descend verticalement vers la sortie à récompense **+1** en bas à droite.
- Il **évite l'état terminal à -3** (en haut à droite), ce qui est un comportement intelligent et optimal.
- La dernière action **0** à l'état 24 est arbitraire, car c'est un **état terminal**.

Analyse des valeurs d'état (V_{opt})

```
[0.93  0.94  0.95  0.96  0.00
0.94  0.95  0.96  0.97  0.98
0.95  0.96  0.97  0.98  0.99
0.96  0.97  0.98  0.99  1.00
0.97  0.98  0.99  1.00  0.00]
```

Ces valeurs montrent que :

- Plus on se rapproche de l'**état terminal positif (24)**, plus la valeur est haute (≈ 1).
- L'état terminal négatif (état 4) a une valeur de 0, car aucune récompense n'est obtenue après son entrée.
- Les états menant vers l'état 4 sont **évités naturellement**, car leurs valeurs sont inférieures à ceux qui mènent à l'état 24.
- **suivre une stratégie stable** qui maximise la récompense cumulée.

Conclusion

La politique optimale trouvée par **Policy Iteration** dirige l'agent vers l'état terminal situé en bas à droite (état 24), qui donne une récompense positive de +1.

En revanche, elle évite l'état terminal en haut à droite (état 4), qui donne une récompense négative de -3.

Cela montre que l'algorithme a correctement appris à maximiser les récompenses à long terme, en exploitant la structure de l'environnement.

Value Iteration

Value Iteration est un algorithme plus rapide que Policy Iteration car il **combine l'évaluation et l'amélioration dans une seule étape**.

À chaque itération, il met à jour la valeur de chaque état $V(s)$ en utilisant directement la **meilleure action possible**, selon la **formule de Bellman optimalité** :

$$V(s) \leftarrow \max_a \sum_{s'} p(s, a, s', r) \cdot [r + \gamma V(s')]$$

L'algorithme s'arrête lorsque les valeurs ne changent presque plus. On en déduit ensuite la politique optimale en choisissant, pour chaque état, l'**action qui maximise cette valeur**.

Expérimentation : Value Iteration sur GridWorld

Politique optimale (issue de value iteration) :

[1 1 1 3 0 1 1 1 1 3 1 1 1 1 3 1 1 1 1 3 1 1 1 1 0]

Valeur des états (issue de value iteration) :

[0.93206535	0.94148015	0.95099005	0.96059601	0.	0.94148015
0.95099005	0.96059601	0.970299	0.9801	0.95099005	0.96059601
0.970299	0.9801	0.99	0.96059601	0.970299	0.9801
0.99	1.	0.970299	0.9801	0.99	1.
0.					

- La politique optimale issue de Value Iteration est exactement la même que celle obtenue avec Policy Iteration, ce qui confirme la cohérence des deux méthodes.
- Les valeurs les plus élevées (~ 1) apparaissent près de l'état terminal positif en bas à droite.
- Les valeurs diminuent progressivement en s'éloignant de cet objectif, ce qui est cohérent avec un facteur de discount $\gamma = 0.99$.
- Les actions 1 (droite $\rightarrow$) et 3 (bas $\downarrow$) dominent, ce qui indique que l'agent cherche à atteindre la sortie positive.
- L'état terminal négatif (état 4, en haut à droite) est **évit**é par la politique, montrant que l'agent apprend à **minimiser les punitions**.

Conclusion

L'algorithme **Value Iteration** parvient à apprendre une politique optimale équivalente à celle de **Policy Iteration**, mais avec une convergence plus directe.

La fonction de valeur obtenue reflète parfaitement la dynamique de l'environnement : des valeurs croissantes vers l'état terminal positif et des choix stratégiques pour éviter les états défavorables.

2. Méthodes Monte Carlo

Monte Carlo Exploring Starts ES

Monte Carlo ES repose sur l'idée de **démarrages aléatoires** (exploring starts), permettant d'assurer une **exploration complète de toutes les paires état-action** en débutant chaque épisode dans un état et une action choisis aléatoirement.

Après chaque épisode **complet**, on met à jour la fonction de valeur d'action $Q(s,a)$ en **moyennant les récompenses cumulées observées** pour cette paire.

Ensuite, on **améliore la politique** en choisissant, pour chaque état, l'action qui maximise $Q(s,a)$

, Cette méthode est **garantie de converger vers la politique optimale**, à condition que **toutes les paires état-action soient explorées**.

Elle s'applique à tout environnement **épisodique** (avec une fin d'épisode bien définie), où **les récompenses peuvent être accumulées sur des trajectoires complètes**, et où l'on peut forcer un démarrage aléatoire.

Expérimentation : Monte Carlo ES sur GridWorld

Paramètres d'entrée utilisés :

- `episodes_count = 100000`
- `gamma = 0.99`

```

Monte Carlo ES: 100%|██████████| 100000/100000 [3:10:15<00:00, 8.76it/s]
Politique optimale estimée par Monte Carlo (action par état) :
[3 0 0 0 0 3 3 3 3 3 3 3 3 3 3 3 3 3 3 1 1 1 1 0]

Valeurs estimées Q(s,a) :
[[-0.94724894 -1.09957323 -0.945159 -0.86083933]
 [-0.94843753 -1.38908154 -1.07675424 -0.95720788]
 [-1.06270122 -1.94088796 -1.33531011 -1.11940423]
 [-1.32286826 -3. -1.90116224 -1.37720696]
 [ 0. 0. 0. 0. ]
 [-0.8666116 -0.95494401 -0.97517445 -0.72769468]
 [-0.87340481 -1.1367448 -1.09833985 -0.7545236 ]
 [-0.94171084 -1.40660746 -1.37350126 -0.76629863]
 [-1.09127205 -1.75342544 -1.9225579 -0.80062247]
 [-1.36916207 -1.72677071 -3. -0.85249337]
 [-0.71535949 -0.75726436 -0.88588969 -0.57865301]
 [-0.72391291 -0.79157883 -0.97160493 -0.53586064]
 [-0.73782975 -0.81123878 -1.11112522 -0.43959039]
 [-0.73895453 -0.84612145 -1.38660279 -0.24406392]
 [-0.7940772 -0.85054292 -1.75460324 -0.03128849]
 [-0.57009878 -0.53899081 -0.73504703 -0.48278775]
 [-0.57526039 -0.44847099 -0.74106084 -0.40203016]
 [-0.52652614 -0.25638122 -0.76212745 -0.19780768]
 [-0.42151991 -0.0234219 -0.7826506 0.21390481]
 [-0.27111115 -0.03324965 -0.85322927 1. ]
 [-0.4803181 -0.40866219 -0.58050891 -0.47234744]
 [-0.48683834 -0.19884178 -0.53960932 -0.39923334]
 [-0.39321059 0.21237011 -0.4123802 -0.19903758]
 [-0.17372869 1. -0.24265288 0.22273086]
 [ 0. 0. 0. 0. ]]

```

Politique optimale estimée

La politique optimale estimée, qui associe à chaque état l'action offrant la meilleure espérance de retour cumulatif, est la suivante :

```
[3 0 0 0 0 3 3 3 3 3 3 3 3 3 3 3 3 3 3 1 1 1 1 0]
```

- ◆ L'agent suit une trajectoire vers le **terminal positif** (état 24) situé en bas à droite.
- ◆ Il **évite l'état 4** (haut droite) qui mène à une **récompense négative (-3)**.
- ◆ Le dernier état étant terminal, son action (0) est arbitraire.

Cette politique indique une préférence pour les mouvements vers le bas (action 3) et vers la gauche (action 0) dans la plupart des états initiaux, tandis qu'en approchant des états terminaux, les actions s'orientent vers la droite (action 1) ou restent statiques dans l'état terminal.

Valeurs Q(s, a) estimées

Le tableau des valeurs Q donne, pour chaque couple état-action, une estimation de la qualité à long terme de choisir cette action. Les observations clés sont :

- Les valeurs négatives ou proches de zéro pour certains états reflètent la pénalité de -3 associée au terminal en haut à droite.
- Les valeurs proches de 1 correspondent aux actions qui conduisent vers l'état terminal positif en bas à droite.
- Ces estimations corroborent la politique optimale, confirmant que l'algorithme a correctement identifié les actions maximisant le retour espéré.

Pourquoi Monte Carlo ES est très lent ?

1. **Basé sur des épisodes complets** : Contrairement aux méthodes de programmation dynamique (comme value iteration), Monte Carlo ne met à jour ses valeurs qu'à la fin d'un épisode complet
2. **Exploration aléatoire** : l'algorithme force l'exploration de toutes les paires (état, action) en démarrant aléatoirement dans chaque épisode, mais certaines trajectoires mènent à des états peu informatifs ou loin des terminaux, ce qui ralentit l'apprentissage utile.
3. **Apprentissage par moyenne lente** : L'algorithme met à jour les valeurs $Q(s,a)$ en prenant une moyenne des retours observés ; mais plus une paire (s,a) est visitée, plus chaque nouvelle mise à jour a peu d'impact, ce qui rend la convergence très lente et explique le temps d'exécution élevé.

Conclusion

L'algorithme **Monte Carlo ES** permet d'estimer une politique optimale dans un environnement inconnu, en s'appuyant sur des trajectoires complètes avec exploration aléatoire.

Le comportement appris est **cohérent avec les récompenses** : aller vers l'état à +1, éviter l'état à -3.

Ce résultat montre que **même sans modèle** (pas besoin des probabilités de transition), on peut converger vers une bonne politique grâce à l'exploration et à la moyenne des retours.

Cependant, cette méthode utilise une moyenne incrémentale pour mettre à jour les valeurs Q, ce qui rend chaque mise à jour moins influente à mesure que le nombre de visites augmente.

Cela explique la **convergence lente** et la **durée d'exécution importante** observées, un comportement typique des méthodes Monte Carlo nécessitant beaucoup d'épisodes pour stabiliser leurs estimations.

3. Apprentissage par différence temporelle (TD Learning)

Q-Learning

L'algorithme **Q-learning** est une méthode d'apprentissage par renforcement **off-policy**, où l'agent apprend une fonction de valeur d'action $Q(s,a)$ qui estime la récompense cumulée maximale qu'il peut espérer en prenant l'action a dans l'état s puis en suivant la meilleure politique possible.

À chaque épisode, l'agent explore l'environnement en choisissant parfois des actions aléatoires (exploration) et d'autres fois en exploitant ses connaissances actuelles (exploitation). Après chaque action, il met à jour la table Q en se basant sur la récompense reçue et la meilleure valeur Q possible de l'état suivant.

Les paramètres clés sont :

- le taux d'apprentissage α , qui contrôle la vitesse de mise à jour des valeurs,
- le facteur d'actualisation γ , qui valorise les récompenses futures,
- et le taux d'exploration ϵ , qui équilibre exploration et exploitation.

Expérimentation : Q-learning sur GridWorld

Paramètres de l'algorithme Q-learning

- **Nombre d'épisodes d'apprentissage (episodes_count) : 100 000**
Nombre total d'épisodes pendant lesquels l'agent explore et apprend.
- **Taux d'exploration (ϵ - epsilon) : 0,1**
Probabilité de choisir une action aléatoire pour favoriser l'exploration (10 % des actions sont choisies aléatoirement).
- **Facteur d'actualisation (γ - gamma) : 0,99**
Importance accordée aux récompenses futures par rapport aux récompenses immédiates.
- **Taux d'apprentissage (α - alpha) : 0,1**
Vitesse à laquelle les nouvelles informations (récompenses et valeurs futures) mettent à

jour les valeurs Q existantes.

```
Q-Learning in progress: 100%|██████████| 100000/100000 [00:09<00:00, 11012.83it/s]
Politique optimale apprise par Q-learning (action par état) :
[1 1 1 3 0 1 1 1 3 3 1 1 1 3 1 1 1 3 1 1 1 0]

Valeurs Q finales (Q-table) :
[[ 0.92274469  0.93206535  0.92274469  0.93206535]
 [ 0.92274469  0.94148015  0.93206535  0.94148015]
 [ 0.93206535  0.95099005  0.94148015  0.95099005]
 [ 0.94148015 -3.          0.95099005  0.96059601]
 [ 0.          0.          0.          0.          ]
 [ 0.93206535  0.94148015  0.92274469  0.94148015]
 [ 0.93206535  0.95099005  0.93206535  0.95099005]
 [ 0.94148015  0.96059601  0.94148015  0.96059601]
 [ 0.95099005  0.95353695  0.95099005  0.970299   ]
 [ 0.96059601  0.93794267 -2.93811082  0.97213136]
 [ 0.94148015  0.95099005  0.93206535  0.95099005]
 [ 0.94148015  0.96059601  0.94148015  0.96059601]
 [ 0.95099005  0.970299   0.95099005  0.970299   ]
 [ 0.96059601  0.9801      0.96059601  0.9801      ]
 [ 0.970299    0.9801      0.95297629  0.99          ]
 [ 0.95099005  0.96059601  0.94148015  0.96059601]
 [ 0.95099005  0.970299   0.95099005  0.970299   ]
 [ 0.96059601  0.9801      0.96059601  0.9801      ]
 [ 0.970299    0.99        0.970299    0.99          ]
 [ 0.9801       0.99        0.9801       1.          ]
 [ 0.94694783  0.970299    0.94616966  0.95215666]
 [ 0.96059601  0.9801      0.96059601  0.970299   ]
 [ 0.970299    0.99        0.970299    0.9801      ]
 [ 0.9801       1.          0.9801       0.99          ]
 [ 0.          0.          0.          0.          ]]
```

```
[→ → → ↓ x
 → → → ↓ ↓
 → → → ↓ ↓
 → → → ↓ ↓
 → → → → x]
```

- La politique optimale guide l'agent vers le coin bas-droit de la grille (récompense +1), tout en évitant l'état terminal en haut à droite (récompense négative -3).
- Les valeurs Q reflètent bien cette stratégie : valeurs plus élevées vers les états proches du terminal positif, valeurs basses ou négatives près du terminal négatif.
- La convergence est rapide : ici, 100 000 épisodes suffisent pour apprendre une politique stable et cohérente.
- Le taux d'exploration $\epsilon = 0,1$ permet un bon équilibre entre exploration et exploitation pendant l'apprentissage.
- La fonction Q capture efficacement la qualité des actions dans chaque état, facilitant l'amélioration progressive de la politique.

Conclusion

Le Q-learning permet à l'agent d'apprendre une politique optimale en interaction directe avec l'environnement, sans connaissance préalable des probabilités de transition, ce qui le rend adapté aux environnements complexes ou partiellement connus.

Les résultats confirment que l'agent a correctement appris à naviguer dans la grille vers les états souhaités en maximisant les récompenses cumulées.

Bien sûr, voici un exemple de texte que tu peux mettre dans ton rapport pour comparer tous ces algorithmes, leurs forces, faiblesses, et particularités sur ton problème GridWorld :